

Lista Avaliativa 5 - Respostas

1)

Não é viável na prática. O algoritmo ótimo exige conhecer antecipadamente todas as futuras referências a páginas, o que é impossível de determinar com certeza. Uma ferramenta de análise estática não consegue prever o comportamento dinâmico de programas que dependem de entrada de dados, condições de execução e decisões em tempo real. O algoritmo ótimo serve apenas como referência teórica para comparar outros algoritmos.

2)

O NRU usa dois bits: R (referenciado) e M (modificado). A cada referência, o bit R é setado. Quando há escrita, o bit M é setado. Periodicamente o sistema limpa os bits R. As páginas são classificadas em quatro classes: (0) não referenciada, não modificada; (1) não referenciada, modificada; (2) referenciada, não modificada; (3) referenciada, modificada. O algoritmo remove uma página aleatória da classe mais baixa não vazia.

Exemplo: temos páginas A(R=0,M=0), B(R=0,M=1), C(R=1,M=0), D(R=1,M=1). A página A está na classe 0, B na classe 1, C na classe 2, D na classe 3. O NRU escolheria A para remoção. Se A não existisse, escolheria B.

3)

Matriz LRU: $n \times n$ bits onde $n=4$

Regra de atualização:

- Quando página k é referenciada:
 - Seta todos os bits da linha k para 1
 - Zera todos os bits da coluna k

Sequência de referência: 3, 3, 2, 1, 2, 3, 0

Estado inicial (todas não referenciadas):

	0	1	2	3
0	0	0	0	0
1	0	0	0	0
2	0	0	0	0
3	0	0	0	0

Após referência 3:

	0	1	2	3
0	0	0	0	0

1		0	0	0	0
2		0	0	0	0
3		1	1	1	0

Após segunda referência 3:

		0	1	2	3
		+-----			
0		0	0	0	0
1		0	0	0	0
2		0	0	0	0
3		1	1	1	0

Após referência 2:

		0	1	2	3
		+-----			
0		0	0	0	0
1		0	0	0	0
2		1	1	0	1
3		1	1	0	0

Após referência 1:

		0	1	2	3
		+-----			
0		0	0	0	0
1		1	0	1	1
2		1	0	0	1
3		1	0	0	0

Após segunda referência 2:

		0	1	2	3
		+-----			
0		0	0	0	0
1		1	0	0	1
2		1	1	0	1
3		1	0	0	0

Após segunda referência 3:

	0	1	2	3
	+-----			
0	0	0	0	0
1	1	0	0	0
2	1	1	0	0
3	1	1	1	0

Após referência 0:

	0	1	2	3
	+-----			
0	0	1	1	1
1	0	0	0	0
2	0	1	0	0
3	0	1	1	0

Página para substituição: Página 1 (menor valor = 0 = menos recentemente usada)

4)

O algoritmo PFF, que significa Frequência de Falta de Página, é basicamente uma estratégia que o sistema operacional usa para decidir dinamicamente quanta memória cada processo deve ter, especialmente quando se usa uma política de alocação global. Em vez de focar em qual página específica remover, o PFF se preocupa em saber se um processo precisa de mais memória ou se está com memória sobrando.

A ideia principal por trás do PFF é que, geralmente, se você der mais quadros de página para um programa, a frequência com que ele tem faltas de página (ou *page faults*) vai diminuir. O algoritmo explora isso monitorando constantemente essa frequência para cada processo e a comparando com dois limites definidos pelo sistema:

- Um limite superior, que representa uma taxa de faltas de página alta demais.
- Um limite inferior, que representa uma taxa de faltas de página muito baixa.

O objetivo é manter a taxa de faltas de cada processo entre esses dois limites. Se um processo começa a ter muitas faltas de página, ultrapassando o limite superior, é um sinal claro de que ele não tem memória suficiente para o seu conjunto de trabalho atual e está entrando em *thrashing*. Nesse momento, o PFF age e aloca mais quadros de página para esse processo, para que ele possa trabalhar de forma mais eficiente.

Por outro lado, se um processo está rodando com uma frequência de faltas de página muito baixa, caindo abaixo do limite inferior, o PFF entende que ele provavelmente tem mais memória do que realmente precisa. Para otimizar o uso dos recursos do sistema, ele remove alguns quadros de página desse processo e os devolve para um *pool* de memória livre, de onde podem ser alocados para outros processos que estejam precisando mais.

Por exemplo, imagine um editor de vídeo que começa uma renderização. Ele vai precisar acessar muitos dados novos, e sua taxa de faltas de página pode disparar, passando do limite superior. O PFF perceberia isso e daria mais memória ao editor. Com mais frames, ele conseguiria manter mais páginas na RAM e sua

taxa de faltas cairia para um nível aceitável. Quando a renderização termina e o programa fica ocioso, sua taxa de faltas de página cairia drasticamente, ficando abaixo do limite inferior. O PFF então agiria de novo, mas dessa vez para pegar de volta a memória que não está mais sendo usada, deixando-a disponível para o resto do sistema.

No fim, o PFF funciona como um gerenciador dinâmico, tentando dar a cada processo a quantidade certa de memória para que ele rode bem, sem sofrer por falta de recursos e sem desperdiçar memória.

5)

O sistema MULTICS utiliza um modelo de memória virtual que combina segmentação com paginação. Essa abordagem permite que cada programa tenha um espaço de endereçamento bidimensional, formado por múltiplos segmentos, e cada um desses segmentos é dividido em páginas de tamanho fixo.

A tradução de um endereço virtual MULTICS para um endereço físico na máquina ocorre através de um processo de múltiplos passos, que envolve o uso de descritores de segmento e tabelas de páginas.

Estrutura do Endereço Virtual

Um endereço virtual no MULTICS é composto por:

1. **Número do Segmento:** Identifica o segmento do programa sendo acessado.
2. **Endereço dentro do Segmento (Deslocamento):** Subdividido em:
 - **Número da Página:** Indica a página dentro do segmento.
 - **Deslocamento dentro da Página:** Indica a palavra exata dentro da página.

Processo de Tradução

O processo de conversão, conforme ilustrado na Figura 3.36 do livro *Sistemas Operacionais Modernos* 4ª edição, pode ser descrito nos seguintes passos:

1. **Localização do Descritor de Segmento:** O hardware utiliza o número do segmento do endereço virtual como um índice para acessar a Tabela de Segmentos do programa. Cada entrada nessa tabela é um Descritor de Segmento. Esse descritor contém informações vitais sobre o segmento, como seu tamanho, permissões de acesso (leitura, escrita, execução) e, o mais importante, o endereço físico de sua Tabela de Páginas, caso o segmento esteja na memória principal.
2. **Acesso à Tabela de Páginas:** Com o endereço da Tabela de Páginas obtido a partir do descritor, o sistema agora pode localizá-la na memória. O número da página do endereço virtual é então usado como um índice para encontrar a entrada correta dentro dessa tabela.
3. **Obtenção do Quadro de Página Físico:** A entrada da tabela de páginas contém o endereço do quadro de página correspondente na memória física. Ela também possui bits de controle, como o bit "Presente/Ausente", que indica se a página está de fato carregada na memória. Se não estiver, ocorre uma *falta de página* (page fault).
4. **Cálculo do Endereço Físico Final:** Finalmente, o endereço do quadro de página (obtido no passo anterior) é combinado com o deslocamento dentro da página (do endereço virtual original) para formar o endereço físico final e definitivo da palavra de memória que se deseja acessar.

Aceleração com o TLB

O processo descrito acima exigiria múltiplas leituras da memória para cada instrução, o que seria extremamente lento. Para resolver isso, o MULTICS foi um dos primeiros sistemas a implementar um hardware de cache especial chamado TLB.

O TLB armazena as traduções de endereço (página virtual para quadro de página físico) mais recentes. O processo real, então, funciona assim:

- Quando uma tradução é necessária, o hardware primeiro verifica o TLB.
- Se a tradução estiver no TLB (TLB Hit): O endereço do quadro físico é obtido instantaneamente, sem a necessidade de consultar as tabelas na memória.
- Se não estiver no TLB (TLB Miss): O hardware executa o processo completo de 4 passos descrito acima, e o resultado da tradução é então armazenado no TLB para acelerar acessos futuros à mesma página.

6)

```
#define _GNU_SOURCE
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sched.h>
#include <string.h>
#include <errno.h>
#include <time.h>

typedef struct
{
    int thread_id;
    int *vector;
    int vector_size;
    int num_cores;
} thread_args_t;

// Função de comparação para o qsort
int compare_integers(const void *a, const void *b)
{
    return (*(int *)a - *(int *)b);
}

void *sort_thread_function(void *args)
{
    thread_args_t *thread_data = (thread_args_t *)args;

    // 1. DEFINE A AFINIDADE DA THREAD

    // Calcula em qual core esta thread deve ser executada.
    int core_id = thread_data->thread_id % thread_data->num_cores;
```

```

    cpu_set_t cpuset;
    CPU_ZERO(&cpuset);
    CPU_SET(core_id, &cpuset);

    if (sched_setaffinity(0, sizeof(cpu_set_t), &cpuset) == -1)
    {
        fprintf(stderr, "Erro ao definir a afinidade da thread %d para o
core %d: %s\n",
            thread_data->thread_id, core_id, strerror(errno));
        pthread_exit(NULL);
    }

    printf("Thread %d foi definida para executar no Core %d.\n",
thread_data->thread_id, core_id);

    // 2. VERIFICA A AFINIDADE

    cpu_set_t affinity_mask;
    CPU_ZERO(&affinity_mask);

    if (sched_getaffinity(0, sizeof(cpu_set_t), &affinity_mask) == -1)
    {
        fprintf(stderr, "Erro ao verificar a afinidade da thread %d: %s\n",
            thread_data->thread_id, strerror(errno));
    }
    else
    {
        if (CPU_ISSET(core_id, &affinity_mask))
        {
            printf("VERIFICAÇÃO: Thread %d confirmada no Core %d.\n",
thread_data->thread_id, core_id);
        }
        else
        {
            printf("VERIFICAÇÃO FALHOU: Thread %d não está no Core %d.\n",
thread_data->thread_id, core_id);
        }
    }

    // 3. EXECUTA A ORDENAÇÃO

    printf("Thread %d iniciando a ordenação do vetor de %d elementos.\n",
        thread_data->thread_id, thread_data->vector_size);

    qsort(thread_data->vector, thread_data->vector_size, sizeof(int),
compare_integers);

    printf("Thread %d terminou a ordenação.\n", thread_data->thread_id);

    pthread_exit(NULL);
}

int main(int argc, char *argv[])
{

```

```

if (argc != 2)
{
    fprintf(stderr, "Uso: %s <tamanho_do_vetor>\n", argv[0]);
    return 1;
}

int n = atoi(argv[1]);
if (n <= 0)
{
    fprintf(stderr, "0 tamanho do vetor deve ser um número
positivo.\n");
    return 1;
}

const int NUM_THREADS = 4;
int *vectors[NUM_THREADS];
pthread_t threads[NUM_THREADS];
thread_args_t thread_args[NUM_THREADS];

// Obtém o número de cores
int num_cores = sysconf(_SC_NPROCESSORS_ONLN);
if (num_cores < 1)
{
    fprintf(stderr, "Não foi possível determinar o número de
cores.\n");
    return 1;
}
printf("Sistema detectado com %d cores.\n\n", num_cores);

srand(time(NULL));

// Aloca e preenche os 4 vetores com números aleatórios
for (int i = 0; i < NUM_THREADS; i++)
{
    vectors[i] = (int *)malloc(n * sizeof(int));
    if (vectors[i] == NULL)
    {
        perror("Falha ao alocar memória para o vetor");
        return 1;
    }
    for (int j = 0; j < n; j++)
    {
        vectors[i][j] = rand() % 10000; // Números entre 0 e 9999
    }
}

for (int i = 0; i < NUM_THREADS; i++)
{
    thread_args[i].thread_id = i;
    thread_args[i].vector = vectors[i];
    thread_args[i].vector_size = n;
    thread_args[i].num_cores = num_cores;
}

```

```
        if (pthread_create(&threads[i], NULL, sort_thread_function,
&thread_args[i]) != 0)
        {
            perror("Falha ao criar a thread");
            return 1;
        }
    }

    for (int i = 0; i < NUM_THREADS; i++)
    {
        pthread_join(threads[i], NULL);
    }

    for (int i = 0; i < NUM_THREADS; i++)
    {
        free(vectors[i]);
    }

    printf("\nTodos os vetores foram ordenados. Programa finalizado.\n");

    return 0;
}
```